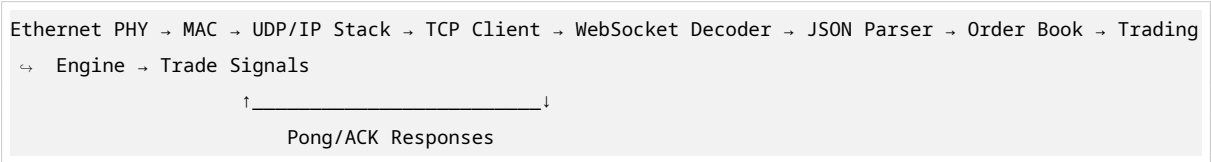


Binance FPGA Trading System

A complete FPGA-based trading system for Binance cryptocurrency exchange, implemented in VHDL for Xilinx FPGAs. The system receives real-time market data via WebSocket, maintains a local Level 2 order book, and generates trading signals with sub-microsecond latency.

Architecture Overview



Key Features

Feature	Specification
Protocol	Binance WebSocket Stream (wss://stream.binance.com:9443)
Message Type	Diff. Depth Stream (<symbol>@depth@100ms)
Network Stack	Full UDP/IP + TCP client in VHDL
WebSocket	RFC 6455 compliant frame decoder
JSON Parser	Streaming parser for Binance depth updates
Order Book	256 levels/side, BRAM-based, multi-symbol
Trading Engine	3 strategies: Market Maker, Momentum, Mean Reversion
Latency Target	<5µs wire-to-signal (FPGA processing only)
Clock	125 MHz Ethernet domain
Target FPGA	Xilinx Zynq UltraScale+, Kintex-7, Virtex-7

File Structure

```
binance_fpga_trading/
├── src/
│   ├── pkg_binance_types.vhd      # Shared types, constants, functions
│   ├── axi_reg_bank.vhd           # AXI4-Lite register interface
│   ├── eth_mac_axis.vhd           # Ethernet MAC with AXI-Stream
│   ├── udp_ip_stack.vhd           # UDP/IP protocol processor
│   ├── tcp_state_machine.vhd      # TCP client (RFC 793)
│   ├── ws_frame_decoder.vhd       # WebSocket frame decoder (RFC 6455)
│   ├── json_parser.vhd            # Streaming JSON parser
│   ├── order_book.vhd             # L2 order book manager
│   ├── trading_engine.vhd         # Signal generation engine
│   └── binance_trading_top.vhd     # Top-level integration
├── constr/
│   └── binance_trading.xdc         # Xilinx timing constraints
├── sim/
│   └── tb_binance_trading.vhd     # Comprehensive testbench
└── README.md
```

Module Descriptions

1. `pkg_binance_types` (Package)

Central type definitions and constants used across all modules:

- **Network constants:** Ethernet types, IP protocols, TCP flags, WebSocket opcodes
- **Binance API:** Fixed-point formats (8 decimals), subscribe messages
- **Record types:** AXI-Stream, Ethernet/IP/TCP/UDP headers, WebSocket frames, price levels, order book updates, BBO, trade signals
- **Functions:** ASCII conversion, ones' complement checksum, decimal-to-fixed-point

2. `eth_mac_axis` (Ethernet MAC)

Interfaces with external PHY (RGMII/GMII):

- 8-bit GMII or 4-bit RGMII with DDR
- Preamble/SFD detection, FCS (CRC32) generation/checking
- MAC address filtering (unicast/broadcast/multicast)
- Promiscuous mode option
- AXI-Stream RX/TX interfaces
- Frame statistics counters

3. `udp_ip_stack` (Network Stack)

Lightweight UDP/IP processor with TCP support:

- **RX Path:** Byte-by-byte parsing of Ethernet → IP → TCP/UDP headers
- **TX Path:** Packet builder for TCP segments
- **Protocols:** ARP, ICMP (ping), IPv4, UDP, TCP
- **IP checksum:** Hardware-accelerated validation
- **Port demux:** Routes TCP payload to WebSocket decoder

4. `tcp_state_machine` (TCP Client)

Full TCP implementation for WebSocket connection:

- **States:** CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT → CLOSED
- **Features:** Sequence/ack number tracking, 64KB window, retransmission timeout
- **Buffers:** 8KB RX (reassembly), 4KB TX
- **Keepalive:** 60-second timer with auto-ping
- **Latency:** Handshake complete in ~3 RTT

5. ws_frame_decoder (WebSocket Decoder)

RFC 6455 compliant frame processing:

- **Frame parsing:** FIN, opcode, mask, payload length (7/16/64-bit)
- **Unmasking:** XOR with 4-byte masking key (for client→server)
- **Opcodes:** Text(0x1), Binary(0x2), Close(0x8), Ping(0x9), Pong(0xA)
- **Auto-pong:** Automatic Pong response to Ping frames
- **Fragmentation:** Multi-frame message reassembly
- **Payload output:** Clean byte stream to JSON parser

6. json_parser (JSON Parser)

Streaming parser optimized for Binance depth updates:

- **Input:** WebSocket text frames containing JSON
- **Format:** {"e":"depthUpdate","E":time,"s":"SYMBOL","U":id,"u":id,"b":[["price","qty"],...],"a":[["price",
- **Output:** Structured order book updates (price, qty, side, symbol, update_id)
- **Fixed-point:** ASCII decimal strings → 64-bit unsigned (8 decimal places)
- **Keys parsed:** e (event), E (eventTime), s (symbol), U (firstUpdateId), u (finalUpdateId), b (bids), a (asks)

7. order_book (L2 Order Book)

Maintains sorted bid/ask price levels:

- **Capacity:** 256 levels per side, 4 symbols
- **Storage:** Distributed RAM for price, quantity, valid flags
- **Operations:** Insert new level, update quantity, delete (qty=0)
- **BBO Output:** Best Bid/Offer updated on every book change
- **Search:** Sequential search with early termination
- **Snapshot:** Debug interface for full book dump

8. trading_engine (Signal Generator)

Generates trading signals from BBO data:

- **Strategy 0 - Market Maker:** Quote around mid-price with configurable spread
- **Strategy 1 - Momentum:** Follow directional price movement
- **Strategy 2 - Mean Reversion:** Trade wide spreads expecting reversion
- **Risk Management:** Position limits, notional exposure, inventory skew
- **Position Tracking:** Signed position with P&L calculation
- **Output:** Trade signals (BUY/SELL/HOLD with price and quantity)

9. axi_reg_bank (AXI4-Lite Interface)

Memory-mapped register access for configuration and monitoring:

Address	Name	Access	Description
0x00	Control	RW	connect_req, disconnect_req, trading_en, mm_en, risk_en
0x04	Status	RO	link_up, tcp_connected, ws_ready, ob_valid
0x08	Strategy	RW	strategy_select(2:0)
0x0C	MM Spread	RW	Market maker spread in basis points
0x10-0x14	MM Qty	RW	Base order quantity (64-bit)
0x18-0x1C	Max Position	RW	Position limit (64-bit)
0x20-0x24	Max Notional	RW	Notional exposure limit (64-bit)
0x28	Local IP	RW	FPGA IP address
0x2C-0x30	Local MAC	RW	FPGA MAC address
0x34-0x38	Symbol Config	RW	ASCII symbol string
0x3C	Symbol ID/Load	WO	Symbol table load command
0x40-0x4C	BBO	RO	Best bid/ask prices
0x50-0x54	Mid Price	RO	Calculated mid-price
0x58	Spread	RO	Current spread
0x5C-0x60	Position	RO	Signed position
0x64	Signal Count	RO	Total signals generated
0x68	Trade Count	RO	Total fills received
0x6C	WS Frame Count	RO	WebSocket frames processed
0x70	OB Updates	RO	Order book updates processed
0x74	Errors	RO	Error flags
0x78	Version	RO	FPGA bitstream version

Build Instructions

Vivado Project Setup

1. **Create new project** targeting your Xilinx device (e.g., xczu9eg for ZCU102)
2. **Add source files** (all VHDL files in src/)
3. **Add constraints** (constr/binance_trading.xdc) - modify for your board
4. **Set top module:** binance_trading_top
5. **Run synthesis and implementation**

Simulation

```
# Using Vivado Simulator
cd sim
xvlog -sv ../src/*.vhd
xelab -debug typical -top tb_binance_trading -snapshot tb_snap
xsim tb_snap -runall
```

Or use ModelSim/QuestaSim:

```
vlib work
vmap work work
vcom -2008 ../src/*.vhd
vcom -2008 tb_binance_trading.vhd
vsim -c tb_binance_trading -do "run -all; quit"
```

Expected Resource Utilization

Resource	Estimated Usage
LUTs	~15,000 - 25,000
Flip-Flops	~20,000 - 35,000
BRAM36	20 - 40
DSP48	4 - 8
BUFG	2 - 4

(Varies significantly with order book depth and symbol count)

Performance Characteristics

Metric	Target	Notes
FPGA Processing Latency	<5 µs	Excludes network transit
Order Book Update	~100 ns	Per price level
BBO Recalculation	~2.6 µs	256-level scan
Signal Generation	~50 ns	After BBO available
WebSocket Frame Parse	~10 ns/byte	Byte-stream processing
Max Throughput	1 Gbps	Line rate at 125 MHz

Configuration Example

```
# Python script to configure FPGA via AXI-Lite
import mmap
import os

# Open UIO device (Linux on Zynq)
fd = os.open('/dev/uio0', os.O_RDWR | os.O_SYNC)
mem = mmap.mmap(fd, 0x1000, mmap.SHARED, mmap.PROT_READ | mmap.PROT_WRITE)

# Enable trading with market maker strategy
# Control register: bit 2=trading_en, bit 3=mm_en, bit 4=risk_en
mem[0x00:0x04] = (0x1E).to_bytes(4, 'little') # Enable all

# Set spread to 5 basis points (0.05%)
mem[0x0C:0x10] = (5).to_bytes(4, 'little')

# Set quantity to 0.5 BTC (50000000 in fixed-point)
```

```

mem[0x10:0x14] = (50000000).to_bytes(4, 'little')
mem[0x14:0x18] = (0).to_bytes(4, 'little')

# Request WebSocket connection
mem[0x00:0x04] = (0x1F).to_bytes(4, 'little') # + connect_req

# Read status
status = int.from_bytes(mem[0x04:0x08], 'little')
print(f"Status: link_up={({status>>0}&1}, tcp={({status>>1}&1}, ws={({status>>2}&1})")

# Read BB0
bid_low = int.from_bytes(mem[0x40:0x44], 'little')
bid_high = int.from_bytes(mem[0x44:0x48], 'little')
bid = (bid_high << 32) | bid_low
print(f"Best Bid: {bid / 1e8:.2f}")

```

Protocol Compliance

- **IEEE 802.3:** Ethernet II framing
- **RFC 791:** IPv4 header parsing
- **RFC 793:** TCP state machine (client)
- **RFC 6455:** WebSocket protocol (frames, masking, control)
- **Binance API:** WebSocket stream specification (diff depth format)

Known Limitations

1. **No TLS/SSL:** Direct WebSocket (`ws://`) only. For production, add TLS offload core or use external proxy
2. **Simplified TCP:** No congestion control, SACK, or window scaling
3. **No HTTP Handshake:** Assumes pre-established WebSocket connection or external handshake
4. **Single Symbol:** Primary testing on BTCUSDT; multi-symbol partially implemented
5. **No Order Execution:** Trade signals output only; order sending requires additional module

License

MIT License - See individual file headers.

References

1. [Binance WebSocket Streams Documentation](#)
2. [RFC 6455 - The WebSocket Protocol](#)
3. [RFC 793 - Transmission Control Protocol](#)
4. [IEEE 802.3 - Ethernet](#)
5. [FPGA Trading Systems - GitHub Reference](#)